

FINAL PROGRESS REPORT

Foundations of AI(CS 5100)

TEAM:

KAVINN PREMMESH TAMILARASU

PRAKASH KUMAR RAMESH

SARVESH ELANCHEZHIAN

Content:

1. [Project Description](#)
2. [Team Roles](#)
3. [Current Progress](#)
 - a. [Component 1](#): Data Collection and Preprocessing
 - b. [Component 2](#): LSTM Baseline - Supervised Learning Approach
 - i. Model Architecture
 - ii. Training
 - iii. Hyperparameter Tuning
 - iv. Test Set Prediction Evaluation
 - c. [Component 3](#): PPO Agent and Gym Environment
 - i. Custom Gymnasium Trading Environment
 - ii. Model Architecture
 - iii. Hyperparameter values
 - iv. Training and Inference
 - v. Hyperparameter Tuning
 - vi. Output Delivered
 - d. [Component 4](#) - Evaluation
 - i. Baseline: Buy-and-Hold
 - ii. Learning-based strategies
 - iii. Alignment and validation
4. [What We Plan to Do Before Final Submission](#)
5. [Project Source Code](#)

Project Description

This project builds and evaluates an automated stock trading system to explore a question that sits at the intersection of machine learning and finance: can a reinforcement learning agent which makes decisions rather than just predictions, outperform both a supervised learning model and a simple buy-and-hold strategy under realistic market conditions?

The system has three main components. The first is a data pipeline. It pulls daily OHLCV data (Open, High, Low, Close, Volume) from Yahoo Finance and enriches it with a standard suite of technical indicators: RSI, MACD (line, signal, and histogram), and both Simple and Exponential Moving Averages over 7, 21, and 50-day windows. The data is split chronologically at 80/20 to avoid any look-ahead bias, producing normalized and raw-price versions of each split for downstream use.

The second component is an LSTM network in PyTorch. It frames trading as a supervised regression task by giving a sliding window of historical features, predicting tomorrow's closing price and converts those predictions into Buy, Sell, or Hold signals using a simple threshold heuristic. The third component is a custom Gymnasium trading environment paired with a PPO agent trained via Stable-Baselines3. Unlike the LSTM, this agent doesn't try to predict prices. It treats trading as a sequential decision problem formulated as a Markov Decision Process: at each step, the agent observes an 18-dimensional state vector (normalized market features plus live portfolio state), picks a discrete action (Buy, Sell, or Hold), and receives a reward proportional to the change in net worth. A 0.1% transaction fee on both buys and sells keeps the incentives grounded in reality.

All three strategies track daily portfolio value starting from a \$10,000 balance, producing a standardized NumPy array that feeds into a shared evaluation harness. Metrics include Cumulative Return, annualized Sharpe Ratio, and Maximum Drawdown, giving a clean side-by-side comparison of passive, predictive, and decision-making approaches to automated trading.

Team Roles:

1. Data Pipeline, baselines and evaluation metrics - **Prakash**
2. LSTM Training, tuning and prediction/strategy evaluation - **Kavinn**
3. RL Environment, PPO training, and policy evaluation - **Sarvesh**

Current Progress:

Component 1: Data Collection and Preprocessing

The primary objective is to develop a reproducible data pipeline that supplies daily market observations and feature-rich inputs for both the supervised LSTM baseline and the reinforcement learning agent, utilizing a chronological train/test split to facilitate rigorous backtesting.

Data source and scope

Historical daily OHLCV (Open, High, Low, Close, Volume) data were retrieved programmatically from Yahoo Finance using the yfinance library. Tickers can be specified individually or via predefined groups (e.g. sector buckets, “Magnificent 7,” union of all presets) so the team can scale experiments across many symbols without manual downloads.

Preprocessing steps

- Alignment and cleaning: Raw downloads are restricted to the OHLCV columns, rows with missing values are dropped, and date indices are normalized for downstream use.
- Technical indicators: For each ticker, the pipeline computes features used as model inputs and/or state: RSI (14-period), MACD (12/26/9), and simple and exponential moving averages at 7, 21, and 50 days. Initial rows with insufficient history for indicators are removed after feature computation.
- Train/test split: A chronological split is applied (default 80% train / 20% test) with no shuffling, so the test period simulates true out-of-sample forecasting and trading.
- Normalization: Min–max scaling is fit only on the training split and applied to both train and test for price-related columns and volume; RSI is scaled to approximately [0, 1] by dividing by 100. Fitted scalers are saved per ticker so predicted values can be mapped back to dollar space when needed.
- Artifacts: For each ticker, the pipeline writes normalized train/test CSVs, raw (dollar) train/test CSVs for execution and reporting, and scaler objects for inverse transforms.

Component 2: LSTM Baseline - Supervised Learning Approach

The LSTM component is my primary responsibility, and it has gone through several stages of development from model definition through training, tuning, and evaluation. It is now fully functional.

Model Architecture:

The core model is a PyTorch module called **LSTMPricePredictor**. It takes a sliding window of consecutive daily feature vectors as input, for example the last 30 or 45 days of normalized features, and outputs a single scalar representing the predicted next day normalized Close price. Internally, the input passes through one or more stacked LSTM layers, and the hidden state from the final timestep is fed through a two layer fully connected head with a ReLU activation in between that produces the prediction. The architecture is flexible so that the hidden dimension, number of LSTM layers, and dropout rate can all be adjusted during experimentation.

A custom PyTorch Dataset class was also written to handle the conversion from the pipeline's output, which is a matrix of T rows by 15 features plus a target vector of T normalized Close values, into overlapping sliding window samples suitable for batched training with a DataLoader.

Training:

The model was trained using Mean Squared Error loss and the Adam optimizer. During training, the pipeline's training CSV was split chronologically into an 80% fitting portion and a 20% validation portion. After each epoch, the model was evaluated on the validation set and the weights from the epoch with the lowest validation loss were saved. This best checkpoint approach prevents overfitting and ensures that the model which generalizes best within the training period is retained. The saved checkpoint includes not just the model weights but also all hyperparameters, the window size, and the full loss history, making it easy to reload and evaluate later.

In the initial training run with default hyperparameters (window size 30, hidden dimension 128, two LSTM layers, dropout 0.2, learning rate 1e-3, batch size 64, 50 epochs), the training MSE dropped sharply in the first few epochs and plateaued near 0.000186, while the validation MSE stabilized in the 0.003 to 0.007 range with the best value of 0.002657 at epoch 46. The training loss curve shows clear convergence, and the gap between training and validation loss indicates mild overfitting, which is a common observation with LSTMs on financial time series.



Hyperparameter Tuning:

To improve on the default configuration, a tuning script was built that performs randomized search over the key hyperparameters. 15 trials were run, each training for 40 epochs, sampling window sizes from {15, 20, 30, 45, 60}, hidden dimensions from {64, 96, 128, 192, 256}, one or two LSTM layers, dropout uniformly between 0.0 and 0.35, learning rates on a log scale between $1e-4$ and roughly $3.5e-3$, and batch sizes from {32, 64, 128}. The best configuration found was a single layer LSTM with hidden dimension 256, window size 45, dropout 0.136, learning rate $2.80e-4$, and batch size 128, achieving a validation MSE of 0.00114 which is a 57% improvement over the default. Across the search, it was observed that larger window sizes and lower learning rates consistently ranked higher, and that single layer LSTMs with moderate dropout frequently outperformed deeper two layer configurations. The full tuning log with all 15 trials and their results has been saved as a CSV file in the repository.

Test Set Prediction Evaluation:

After selecting the best checkpoint from tuning, the model's raw prediction quality was evaluated on the held out test set. The evaluation script constructs sliding windows using the tail of the training data as context for the first test day prediction, then walks forward through the entire test period. On 494 test days, the model achieved an MSE of 0.0085, MAE of 0.071, and RMSE of 0.092 in normalized space. After inverting the MinMaxScaler for the Close column, these translate to approximately \$11.18 MAE and \$14.51 RMSE in dollar terms. Given that AAPL's price during the test window ranges roughly from \$130 to \$250, the dollar MAE represents about 5 to 8 percent of price. This is a reasonable result for a basic daily LSTM, though not precise enough to claim strong forecasting ability. A systematic downward bias was also identified in the predictions where the model tends to regress toward the training period mean

rather than tracking the test period's upward trend, which is a well documented challenge with LSTMs on non stationary data.

Component 3: PPO Agent and Gym Environment

Custom Gymnasium Trading Environment:

Implemented the full step() logic in the custom trading environment, which was the missing one. The environment models a single-asset trading problem as a Markov Decision Process:

- Action Space: Discrete (3) -> Hold (0), Buy (1), Sell (2)
- Buy: Spends the entire available cash balance to purchase shares at the current raw Close price. Transaction fee of 0.1% is applied exactly as in Yang et al. (2020), using the formula $\text{shares} = \text{balance} / (\text{price} * (1 + \text{fee}))$ which guarantees the balance hits exactly zero with no floating-point overspend
- Sell: Sells all held shares at the current raw price, subtracts 0.1% transaction fee, and returns the net proceeds to cash
- Hold: No action taken
- Reward: Normalizing by initial balance scales the reward to a small decimal range which is critical for the value network to learn effectively.
- Termination: Episode ends when the agent reaches the last row of the dataset, or if net worth falls to zero (bankruptcy).

A key decision was accepting both a normalized dataframe for observations and a raw price dataframe for trade execution so that it prevents the agent from using normalized prices (0-1 range) as dollar amounts, which would produce incorrect trade sizes. The 18-dimensional observation vector contains the following: Open, High, Low, Close, Volume, RSI, MACD, MACD_Signal, MACD_Hist, SMA_7, SMA_21, SMA_50, EMA_7, EMA_21, EMA_50, current_balance, shares_held, net_worth.

Model Architecture:

The PPO agent uses a two-headed Multi-Layer Perceptron (MLP) architecture provided by Stable-Baselines3's MlpPolicy, consisting of a shared feature extractor followed by separate Actor and Critic heads. Both the Actor and Critic networks are composed of two fully-connected hidden layers of 256 units each with ReLU activations and are four times larger than the SB3 default of 64 units, chosen to provide sufficient capacity for learning non-trivial trading patterns across a diverse feature set. The Actor head outputs a probability distribution over the three discrete actions (Buy, Sell, Hold), while the Critic head outputs a single scalar estimating the expected cumulative return from the current state. The input to the network is an 18-dimensional observation vector comprising 15 normalized market features (OHLCV, RSI, MACD, and moving averages) and 3 live portfolio state values (current cash balance, shares held, and net worth), all further normalized at runtime via VecNormalize using running mean and standard deviation statistics accumulated during training. The PPO algorithm updates both heads jointly using the

clipped surrogate objective with a clip range of 0.2, GAE-Lambda of 0.95, a discount factor of 0.99, and a learning rate of 1e-4.

Hyperparameter values:

Hyperparameter	Value	Reason
Policy	MlpPolicy	Tabular financial features
Network	[256,256]	4× larger than SB3 default and it is needed for non-trivial pattern learning
Learning rate	1e-4	Reduced from 3e-4 to eliminate policy oscillation
N_steps	2048	Standard rollout buffer size
Gamma	0.99	Standard long-horizon discount
Clip range	0.2	Standard PPO safety threshold
Device	CPU	MlpPolicy trains faster on CPU than GPU due to transfer overhead

Training and Inference:

A complete end-to-end CLI script that handles the full lifecycle of the PPO agent. The training phase loads the processed AAPL data, wraps the environment with DummyVecEnv as required by Stable-Baselines3, and applies VecNormalize to normalize observations at runtime, resolving the scale mismatch between normalized market features and raw portfolio dollar values. The trained model and normalization statistics are saved together, with an automatic timestamped backup created before each run to prevent overwriting a good model. The inference phase loads the saved model, freezes the normalization statistics so no test data leaks into the normalization, and runs the agent deterministically through the 494-day test period, recording daily portfolio values. The final output is saved to outputs/portfolios/ppo.npy alongside a metadata file, ready to be passed into the shared evaluation harness.

Hyperparameter Tuning:

Training was not straightforward. The initial implementation produced explained_variance nearly equal to 0 throughout the entire run, indicating the value network was completely failing to learn. This was traced to following root causes:

Problem 1 was the Raw reward scale. The reward was defined as raw dollar change in net worth (like + or - \$300). A small MLP cannot fit a regression target in the thousands. Dividing by initial_balance brought the reward into the + or - 0.05 range, after which explained_variance immediately rose to 0.93+.

Problem 2 was the Observation scale mismatch. Market features were normalized to [0, 1] by the data pipeline, but portfolio state (balance, shares, net worth) was in raw dollar units (\$10,000+). This made gradient learning unstable. Adding VecNormalize with a running normalizer resolved this.

Problem 3 was Learning rate too high. At $lr=3e-4$ with 2M timesteps, clip_fraction increased from 0.22 to 0.28 over training, which is the opposite of convergence. Reducing to $lr=1e-4$ brought clip_fraction down to 0.08–0.13, indicating stable, controlled policy updates.

Problem 4 was the Overfitting beyond optimal timesteps. Training to 5M timesteps caused the test return to drop from +35.37% to +3.07%. The agent memorized the 1,972-day training trajectory so specifically that it lost generalization. The 2M checkpoint was restored as the submitted model.

Tuning progression:

Run	Learning Rate	Timesteps	Test Return	Explained Variance	Comment
Initial	3e-4	500k	+93%	Nearly 0	-
Fixed reward + VecNorm	3e-4	500k	+14.56%	0.93	Learning
Longer	3e-4	2M	+21.32%	0.94	Oscillating but did not converge
Lower LR	1e-4	2M	+35.37%	0.97	Best result
Overfitting	1e-4	5M	+3.07%	0.91-0.99	Overfitted

Output Delivered:

The output file is a 1D array representing daily portfolio values on the AAPL test set (2023-2024), starting at \$10,000. The Final value was \$13,536.86 (+35.37% cumulative return).

Component 4 - Evaluation:

To evaluate all strategies on a common test window using comparable portfolio value time series and standard risk/return metrics, and to present results in tables and charts suitable for cross-method comparison.

1. **Baseline: Buy-and-Hold:** A buy-and-hold baseline was implemented on the test split using actual close prices from test_raw.csv. The simulator invests the initial cash on the first test day (maximum whole shares plus residual cash) and marks the portfolio to market each subsequent day. The output is a one-dimensional array of daily portfolio values over the test horizon, comparable to the learning-based strategies.
2. **Learning-based strategies**
 - a. *LSTM*: A heuristic trading policy driven by next-day price predictions (implemented by the team's LSTM workflow) produces a daily portfolio path on the same calendar as the test set; transaction costs are applied in the simulation so behavior is closer to the RL setup.
 - b. *PPO*: The RL agent's test-run net worth series is loaded from saved outputs after training/inference for the same ticker and period.
3. **Alignment and validation:** Because different components may emit series of different lengths or different implied starting capital, an evaluation harness was added that:
 - a. Optionally truncates all series to a common length (shortest series) when sources disagree;
 - b. Validates that all included strategies start from the same initial portfolio value within a small tolerance (so a \$100k LSTM run is not compared to a \$10k PPO run by mistake);
 - c. Produces a summary table with cumulative return, annualized Sharpe ratio (from daily portfolio values), and maximum drawdown.

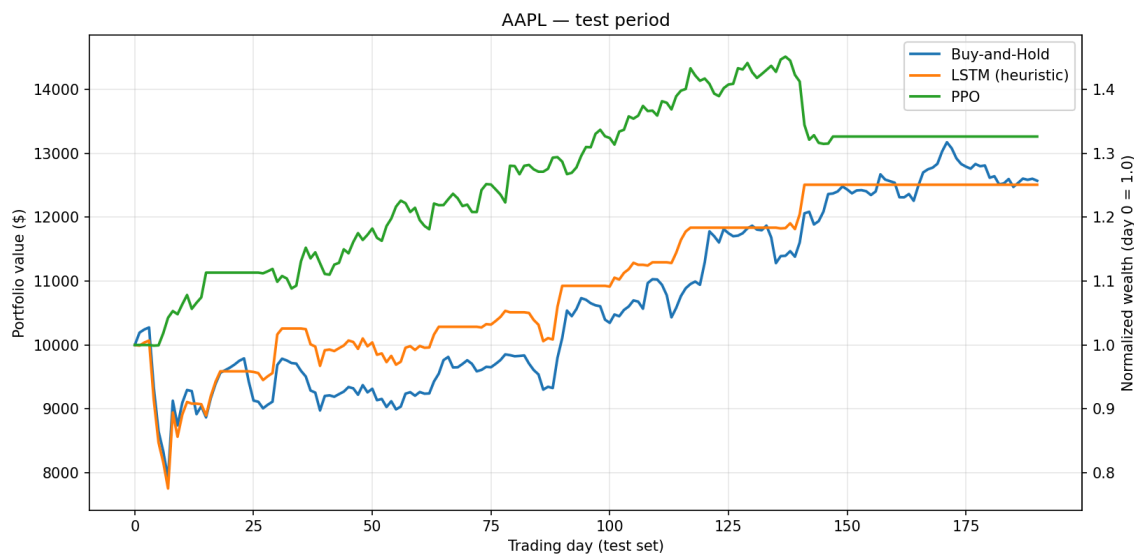


Figure X. Portfolio value over the AAPL test period for Buy-and-Hold, LSTM (heuristic), and PPO.
Note: The left axis shows portfolio value in U.S. dollars; the right axis shows normalized wealth (day 0 = 1.0).

It is clearly visible that the PPO strategy outperformed both the Buy-and-Hold and LSTM benchmarks. It not only avoided the significant early-stage drawdown seen in the other series but also maintained a consistently higher wealth trajectory, ultimately ending the period with superior cumulative returns.

What We Plan to Do Before Final Submission:

Our current results are based on a single stock (AAPL). Before the final submission, we plan to:

1. **Expand to multiple company datasets** — Train and evaluate both the LSTM and PPO models on stock data from several additional companies across different sectors (e.g., technology, healthcare, finance) to test how well each approach generalizes beyond a single ticker.
2. **Comparative analysis across tickers** — Evaluate whether the relative performance of the PPO agent versus the LSTM model holds consistently across different stocks, or whether certain market behaviors favor one approach over the other.
3. **Hyperparameter refinement** — Use insights from the multi-stock evaluation to fine-tune model hyperparameters (LSTM hidden size, sequence length, PPO reward shaping, learning rate) for improved and more robust performance.

Project Source Code:

<https://github.com/Kavinn-0411/Automated-Stock-Trading-Bot>